

tf.slice Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/slice

Extracts a slice from a tensor.

Function Signatures

```
tf.slice( input_, begin, size, name=None )
```

Code Examples

```
tf.slice(  
input_, begin, size, name=None  
)
```

```
tf.slice(  
input_, begin, size, name=None  
)
```

```
t = tf.constant([[[1, 1, 1], [2, 2, 2]],  
                [[3, 3, 3], [4, 4, 4]],  
                [[5, 5, 5], [6, 6, 6]]])  
tf.slice(t, [1, 0, 0], [1, 1, 3]) # [[[3, 3, 3]]]  
tf.slice(t, [1, 0, 0], [1, 2, 3]) # [[[3, 3, 3],  
#   [4, 4, 4]]]  
tf.slice(t, [1, 0, 0], [2, 1, 3]) # [[[3, 3, 3]],  
#   [[5, 5, 5]]]
```

```
t = tf.constant([[[1, 1, 1], [2, 2, 2]],  
                [[3, 3, 3], [4, 4, 4]],  
                [[5, 5, 5], [6, 6, 6]]])  
tf.slice(t, [1, 0, 0], [1, 1, 3]) # [[[3, 3, 3]]]  
tf.slice(t, [1, 0, 0], [1, 2, 3]) # [[[3, 3, 3],  
#   [4, 4, 4]]]  
tf.slice(t, [1, 0, 0], [2, 1, 3]) # [[[3, 3, 3]],  
#   [[5, 5, 5]]]
```

Documentation

Extracts a slice from a tensor.

View aliases

Compat aliases for migration

See

Migration guide for
more details.

`tf.compat.v1.slice`

Used in the notebooks

- Introduction to tensor slicing
 - TFLite Authoring Tool
 - TensorFlow Lite Model Analyzer
 - Client-efficient large-model federated learning via ``federated_select`` and sparse aggregation
 - Graph regularization for sentiment classification using synthesized graphs

See also `tf.strided_slice`.

This operation extracts a slice of size `size` from a tensor `input_` starting at the location specified by `begin`. The slice size is represented as a tensor shape, where `size[i]` is the number of elements of the `i`'th dimension of `input_` that you want to slice. The starting location (`begin`) for the slice is represented as an offset in each dimension of `input_`. In other words, `begin[i]` is the offset into the `i`'th dimension of `input_` that you want to slice from.

Note that `tf.Tensor.getitem` is typically a more pythonic way to perform slices, as it allows you to write `foo[3:7, :-2]` instead of `tf.slice(foo, [3, 0], [4, foo.get_shape()[1]-2])`.

begin is zero-based; size is one-based. If size[i] is -1, all remaining elements in dimension i are included in the slice. In other words, this is equivalent to setting:

$$\text{size}[i] = \text{input_dim_size}(i) - \text{begin}[i]$$

This operation requires that:

$$0 \leq \text{begin}[i] \leq \text{begin}[i] + \text{size}[i] \leq D_i \text{ for } i \text{ in } [0, n]$$

For example:

Returns